
Session 4 (2h) — 인터럽트 (Interrupts)

Covers: 인터럽트

1. Why interrupts exist — ~30 min

Everything so far reads inputs inside `loop()` — but if `loop()` is busy doing something slow (a long sensor read, a long calculation), a fast button press could be missed entirely between checks. An **interrupt** pauses whatever the MCU is doing, instantly runs a small handler function, then resumes exactly where it left off — guaranteeing the event isn't missed.

2. `attachInterrupt()` — ~40 min

On Arduino Uno, only **pins D2 and D3** support external hardware interrupts.

```
attachInterrupt(digitalPinToInterrupt(2), myHandlerFunction, FALLING);
```

- `FALLING` = trigger when the pin goes HIGH→LOW (matches a button with `INPUT_PULLUP`)
- Other modes: `RISING`, `CHANGE`, `LOW`
- The handler function (**ISR — Interrupt Service Routine**) must be short and fast — no `delay()`, no `Serial.print()` inside it, ideally just flag-setting.

3. The `volatile` keyword — ~20 min

Any variable shared between an ISR and the rest of your code must be declared `volatile` — it tells the compiler “this can change at any moment, don't optimize away re-reads of it.” Forgetting `volatile` is the #1 interrupt bug.

```
volatile int pressCount = 0;
void handleButton() { pressCount++; }
```

4. Interrupt-driven debounce — ~20 min

Interrupts can fire multiple times from switch bounce too — use a timestamp check inside the ISR (comparing `millis()`) as a lightweight debounce, since you can't call `delay()` inside an ISR.

Practice: `practice/interrupt_button_counter.ino` — ~10 min

Counts button presses using a hardware interrupt on D2, printing the count over Serial.

Wiring for this session

Component	Pin	Other end
Switch leg A	D2	—
Switch leg B	GND	—

Checkpoint before Session 5

- ☐ Can explain why interrupts exist vs. just polling in `loop()`
- ☐ Can explain what `volatile` does and why it's required here

☐ Button counter increments reliably with no missed or double-counted presses